

Data Science

Lecture 4: Introduction to Pandas

Sam Chan

Semester 1 AY2023/27

Course Outlines

- ▶ Introduction to IPython
- ▶ Introduction to Numpy
- ▶ Data Munging
- ▶ Introduction to Pandas
 - ▶ Pivot table
 - ▶ Working with time-series
 - ▶ High performance Pandas
- ▶ Dimension Reduction
- ▶ Introduction to Scikit-learn
- ▶ Detection of Outliers
- ▶ Testing and Validation
- ▶ Natural Language Processing
- ▶ Social Network Analysis

Introduction to Pandas

Description

- ▶ Pandas is a Python library used for working with data sets.
- ▶ It has functions for analyzing, cleaning, exploring, and manipulating data.
- ▶ The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Use Pandas?

- ▶ Pandas allows us to analyze big data and make conclusions based on statistical theories.
- ▶ Pandas can clean messy data sets, and make them readable and relevant.
- ▶ Relevant data is very important in data science.

Introduction to Pandas

Import

```
import pandas as pd
```

Data Structure

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array, sizeimmutable.
Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size-mutable array.

Introduction to Pandas

Series

```
pandas.Series( data, index, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, list, constants
2	index Index values must be unique and hashable, same length as data. Default np.arange(n) if no index is passed.
3	dtype dtype is for data type. If None, data type will be inferred
4	copy Copy data. Default False

Introduction to Pandas

Object creation

```
In [3]: s = pd.Series([1, 3, 5, np.nan, 6, 8])
```

```
In [4]: s
```

```
Out[4]:
```

```
0    1.0
```

```
1    3.0
```

```
2    5.0
```

```
3    NaN
```

```
4    6.0
```

```
5    8.0
```

```
dtype: float64
```

Introduction to Pandas

Getting

```
In [23]: df["A"]
```

```
Out[23]:
```

```
2013-01-01    0.469112
```

```
2013-01-02    1.212112
```

```
2013-01-03   -0.861849
```

```
2013-01-04    0.721555
```

```
2013-01-05   -0.424972
```

```
2013-01-06   -0.673690
```

```
Freq: D, Name: A, dtype: float64
```

Introduction to Pandas

Setting

```
In [45]: s1 = pd.Series([1, 2, 3, 4, 5, 6], index=pd.date_range("20130102", periods=6))
```

```
In [46]: s1
```

```
Out[46]:
```

```
2013-01-02    1
2013-01-03    2
2013-01-04    3
2013-01-05    4
2013-01-06    5
2013-01-07    6
Freq: D, dtype: int64
```

```
In [47]: df["F"] = s1
```

Introduction to Pandas

Data Frame

```
pandas.DataFrame( data, index, columns, dtype, copy)
```

Sr.No	Parameter & Description
1	data data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame.
2	index For the row labels, the Index to be used for the resulting frame is Optional Default np.arange(n) if no index is passed.
3	columns For column labels, the optional default syntax is - np.arange(n). This is only true if no index is passed.
4	dtype Data type of each column.

Introduction to Pandas

Data Frame

```
import pandas as pd
data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data,columns=['Name','Age'])
print df
```

Its **output** is as follows –

	Name	Age
0	Alex	10
1	Bob	12
2	Clarke	13

Introduction to Pandas

Panel

```
pandas.Panel(data, items, major_axis, minor_axis, dtype, copy)
```

Parameter	Description
data	Data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame
items	axis=0
major_axis	axis=1
minor_axis	axis=2
dtype	Data type of each column
copy	Copy data. Default, false

Introduction to Pandas

Panel -3D

From 3D ndarray

```
# creating an empty panel
import pandas as pd
import numpy as np

data = np.random.rand(2,4,5)
p = pd.Panel(data)
print p
```

Its **output** is as follows –

```
<class 'pandas.core.panel.Panel'>
Dimensions: 2 (items) x 4 (major_axis) x 5 (minor_axis)
Items axis: 0 to 1
Major_axis axis: 0 to 3
Minor_axis axis: 0 to 4
```

Introduction to Pandas

Panel - 3D

```
# creating an empty panel
import pandas as pd
import numpy as np
data = {'Item1' : pd.DataFrame(np.random.randn(4, 3)),
        'Item2' : pd.DataFrame(np.random.randn(4, 2))}
p = pd.Panel(data)
print p['Item1']
```

[Live Demo](#)

Its **output** is as follows –

	0	1	2
0	0.488224	-0.128637	0.930817
1	0.417497	0.896681	0.576657
2	-2.775266	0.571668	0.290082
3	-0.400538	-0.144234	1.110535

We have two items, and we retrieved item1. The result is a DataFrame with 4 rows and 3 columns, which are the **Major_axis** and **Minor_axis** dimensions.

Introduction to Pandas

Pivot Table

- ▶ Create from dataframe
- ▶ Create from file

Pivot Table

Create from dataframe

```
import pandas as pd

# create DataFrame with Team records
dataFrame = pd.DataFrame({'Team ID': {0: 5, 1: 9, 2: 6, 3: 11, 4: 2, 5: 7 },
                          'Team Name...': {0: 'India', 1: 'Australia', 2: 'Bangladesh', 3: 'South Africa',
                          4: 'Sri Lanka', 5: 'England'}, 'Team Points': {0: 95, 1: 93, 2: 42, 3: 60, 4: 80, 5: 55
                          }, 'Team Rank': {0: 'One', 1: 'Two', 2: 'Six', 3: 'Four', 4: 'Three', 5: 'Five'}})

print("\n... Pivot ...")

# multiple columns
print(pd.pivot_table(dataFrame, index = ["Team ID", "Team Name", "Team Rank"]))
```

Pivot Table

Create from dataframe - Output

... Pivot ...

			Team Points
Team ID	Team Name	Team Rank	
2	Sri Lanka	Three	80
5	India	One	95
6	Bangladesh	Six	42
7	England	Five	55
9	Australia	Two	93
11	South Africa	Four	60

Pivot Table

Create from file

```
df = pd.read_excel("../in/sales-funnel.xlsx")  
df.head()
```

	Account	Name	Rep	Manager	Product	Quantity	Price	Status
0	714466	Trantow-Barrows	Craig Booker	Debra Henley	CPU	1	30000	presented
1	714466	Trantow-Barrows	Craig Booker	Debra Henley	Software	1	10000	presented
2	714466	Trantow-Barrows	Craig Booker	Debra Henley	Maintenance	2	5000	pending
3	737550	Fritsch, Russel and Anderson	Craig Booker	Debra Henley	CPU	1	35000	declined
4	146832	Kiehn-Spinka	Daniel Hilton	Debra Henley	CPU	2	65000	won

Pivot Table

Pivot the data

```
pd.pivot_table(df, index=["Name"])
```

	Account	Price	Quantity
Name			
Barton LLC	740150	35000	1.000000
Fritsch, Russel and Anderson	737550	35000	1.000000
Herman LLC	141962	65000	2.000000
Jerde-Hilpert	412290	5000	2.000000
Kassulke, Ondricka and Metz	307599	7000	3.000000
Keeling LLC	688981	100000	5.000000

Pivot Table

Pivot the data

```
pd.pivot_table(df, index=["Name", "Rep", "Manager"])
```

			Account	Price	Quantity
Name	Rep	Manager			
Barton LLC	John Smith	Debra Henley	740150	35000	1.000000
Fritsch, Russel and Anderson	Craig Booker	Debra Henley	737550	35000	1.000000
Herman LLC	Cedric Moss	Fred Anderson	141962	65000	2.000000
Jerde-Hilpert	John Smith	Debra Henley	412290	5000	2.000000
Kassulke, Ondricka and Metz	Wendy Yule	Fred Anderson	307599	7000	3.000000
Keeling LLC	Wendy Yule	Fred Anderson	688981	100000	5.000000

Pivot Table

Index - group and aggregate

```
pd.pivot_table(df, index=["Manager", "Rep"])
```

		Account	Price	Quantity
Manager	Rep			
Debra Henley	Craig Booker	720237.0	20000.000000	1.250000
	Daniel Hilton	194874.0	38333.333333	1.666667
	John Smith	576220.0	20000.000000	1.500000
Fred Anderson	Cedric Moss	196016.5	27500.000000	1.250000
	Wendy Yule	614061.5	44250.000000	3.000000

Pivot Table

Index - identify values column

```
pd.pivot_table(df, index=["Manager", "Rep"], values=["Price"])
```

		Price
Manager	Rep	
Debra Henley	Craig Booker	20000
	Daniel Hilton	38333
	John Smith	20000
Fred Anderson	Cedric Moss	27500
	Wendy Yule	44250

The price column automatically averages the data but we can do a count or a sum. Adding them is simple using **aggfunc** and **np.sum**.

Pivot Table

Index - aggregate and sum

```
pd.pivot_table(df, index=["Manager", "Rep"], values=["Price"], aggfunc=np.sum)
```

		Price
Manager	Rep	
Debra Henley	Craig Booker	80000
	Daniel Hilton	115000
	John Smith	40000
Fred Anderson	Cedric Moss	110000
	Wendy Yule	177000

Pivot Table

Index - aggregate and sum

aggfunc can take a list of functions. Lets try a mean using the numpy **mean** function and **len** to get a count.

```
pd.pivot_table(df, index=["Manager", "Rep"], values=["Price"], aggfunc=[np.mean, len])
```

		mean	len
		Price	Price
Manager	Rep		
Debra Henley	Craig Booker	20000	4
	Daniel Hilton	38333	3
	John Smith	20000	2
Fred Anderson	Cedric Moss	27500	4
	Wendy Yule	44250	4

Time Series

Date / Time

```
import pandas as pd  
pd.to_datetime('2018-01-15 3:45pm')
```

```
Timestamp('2018-01-15 15:45:00')
```

```
pd.to_datetime('7/8/1952')
```

```
Timestamp('1952-07-08 00:00:00')
```

Time Series

Array

```
pd.to_datetime(['2/25/10', '8/6/17', '12/15/12'], format='%m/%d/%y')
```

```
DatetimeIndex(['2010-02-25', '2017-08-06', '2012-12-15'], dtype='datetime64[ns]', freq=None)
```

Time Series

Example

```
opsd_daily = pd.read_csv('opsd_germany_daily.csv')  
opsd_daily.shape
```

```
(4383, 5)
```

4383 rows, 5 columns

Time Series

Check data type

```
opsd_daily.dtypes
```

```
Date datetime64[ns]  
Consumption float64  
Wind float64  
Solar float64  
Wind+Solar float64  
dtype: object
```

Time Series

Index

```
opsd_daily = opsd_daily.set_index('Date')
```

```
opsd_daily.index
```

```
DatetimeIndex(['2006-01-01', '2006-01-02', '2006-01-03', '2006-01-04',  
              '2006-01-05', '2006-01-06', '2006-01-07', '2006-01-08',  
              '2006-01-09', '2006-01-10',  
              ...  
              '2017-12-22', '2017-12-23', '2017-12-24', '2017-12-25',  
              '2017-12-26', '2017-12-27', '2017-12-28', '2017-12-29',  
              '2017-12-30', '2017-12-31'],  
              dtype='datetime64[ns]', name='Date', length=4383, freq=None)
```

Time Series

Use Index

```
opsd_daily.head(3)
```

	Consumption	Wind	Solar	Wind+Solar
Date				
2006-01-01	1069.184	NaN	NaN	NaN
2006-01-02	1380.521	NaN	NaN	NaN
2006-01-03	1442.533	NaN	NaN	NaN

Time Series

Use Index

```
# Add columns with year, month, and weekday name
opds_daily['Year'] = opds_daily.index.year
opds_daily['Month'] = opds_daily.index.month
opds_daily['Weekday Name'] = opds_daily.index.weekday_name
```

Time Series

Time-based Index

Select row

```
opsd_daily.loc['2017-08-10']
```

```
Consumption 1351.49  
Wind 100.274  
Solar 71.16  
Wind+Solar 171.434  
Year 2017  
Month 8  
Weekday Name Thursday  
Name: 2017-08-10 00:00:00, dtype: object
```

Time Series

Use Time-based Index

```
opds_daily.loc['2014-01-20':'2014-01-22']
```

	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday Name
Date							
2014-01-20	1590.687	78.647	6.371	85.018	2014	1	Monday
2014-01-21	1624.806	15.643	5.835	21.478	2014	1	Tuesday
2014-01-22	1625.155	60.259	11.992	72.251	2014	1	Wednesday

Time Series

Visualization

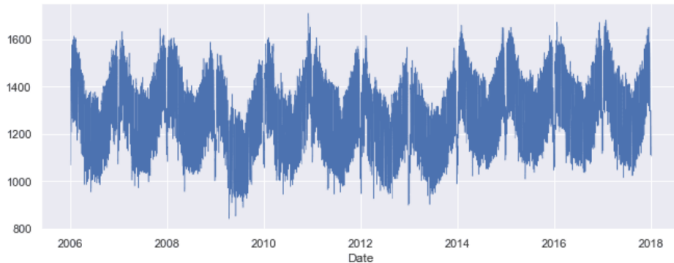
```
import matplotlib.pyplot as plt  
# Display figures inline in Jupyter notebook
```

```
import seaborn as sns  
# Use seaborn style defaults and set the default figure size  
sns.set(rc={'figure.figsize':(11, 4)})
```

Time Series

Visualization

```
opsd_daily['Consumption'].plot(linewidth=0.5);
```



Times Series

Advanced Usage

- ▶ Frequencies
- ▶ Resampling
- ▶ Rolling windows
- ▶ Trend
- ▶ Seasonality

High Performance

Example

```
import pandas as pd
nrows, ncols = 100000, 100
rng = np.random.RandomState(42)
df1, df2, df3, df4 = (pd.DataFrame(rng.rand(nrows, ncols))
                       for i in range(4))
```

To compute the sum of all four `DataFrame`s using the typical Pandas approach, we can just write the sum:

```
%timeit df1 + df2 + df3 + df4
```

10 loops, best of 3: 87.1 ms per loop

High Performance

eval()

The same result can be computed via `pd.eval` by constructing the expression as a string:

```
%timeit pd.eval('df1 + df2 + df3 + df4')
```

10 loops, best of 3: 42.2 ms per loop

The `eval()` version of this expression is about 50% faster (and uses much less memory), while giving the same result:

```
np.allclose(df1 + df2 + df3 + df4,  
            pd.eval('df1 + df2 + df3 + df4'))
```

True

High Performance

query()

The `DataFrame` has another method based on evaluated strings, called the `query()` method. Consider the following:

```
result1 = df[(df.A < 0.5) & (df.B < 0.5)]  
result2 = pd.eval('df[(df.A < 0.5) & (df.B < 0.5)]')  
np.allclose(result1, result2)
```

True

As with the example used in our discussion of `DataFrame.eval()`, this is an expression involving columns of the `DataFrame`. It cannot be expressed using the `DataFrame.eval()` syntax, however! Instead, for this type of filtering operation, you can use the `query()` method:

```
result2 = df.query('A < 0.5 and B < 0.5')  
np.allclose(result1, result2)
```

True

High Performance

Advanced Topics

- ▶ Index Optimization
- ▶ Vectorize Operations
- ▶ Memory Optimization
- ▶ Filter Optimization